

Frame Buffers

Computer Graphics
Instructor: Sungkil Lee

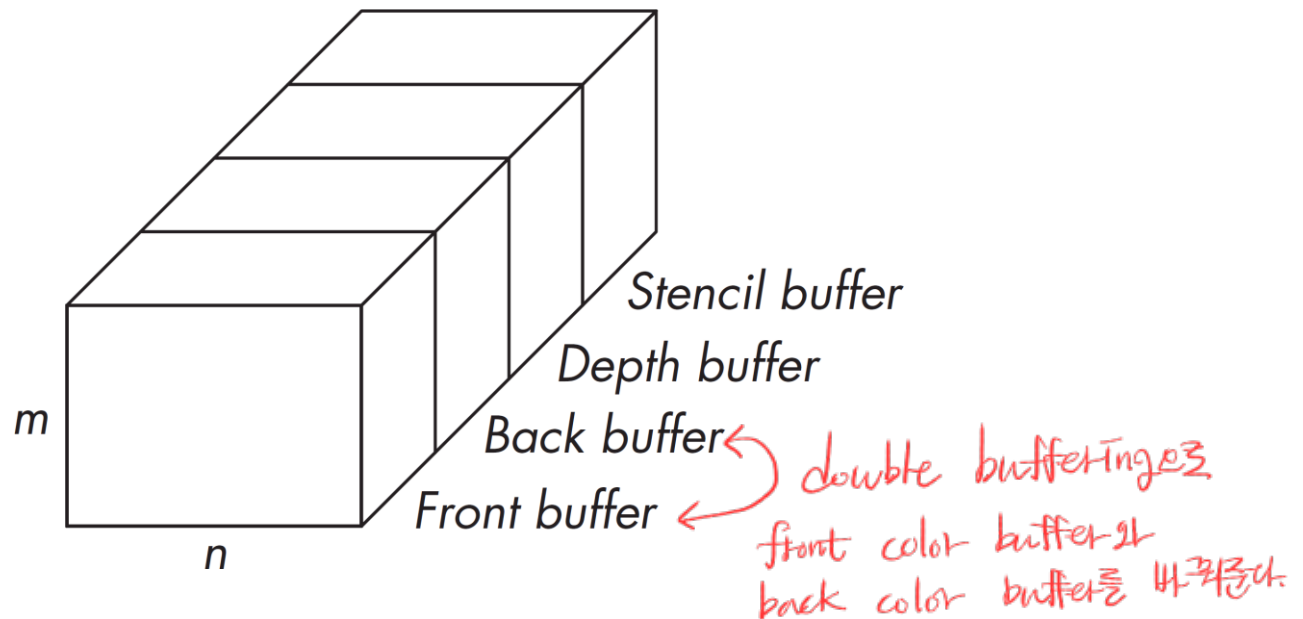
Today

- **Frame buffers**
- **Post-fragment operations and compositing**
 - Depth buffering
 - Blending
 - Compositing
- **Frame Buffer Objects (FBOs)**
 - Frame buffer objects (FBOs)
 - Render-to-Texture
 - Example Applications

Frame Buffers

Frame Buffers

- **Frame buffers store fragments generated via rendering pipeline in large 2D memory areas.**
 - Standard buffers: color buffers and depth buffers
 - Front and back color buffers are defined for double buffering.
 - Additional buffers: stencil buffers
 - In many cases, depth-stencil buffers are used together.



Frame Buffers

- **Frame buffers are defined by resolution and bit depth.**

- Typical bit depths: 8 bpp, 24 bpp, and 32 bpp (RGBA)
- This specification is similar to those of textures. *Image를 표현하는 데 spec 비슷함*
- Hence, textures can be used as frame buffers, and we call such textures as render targets.

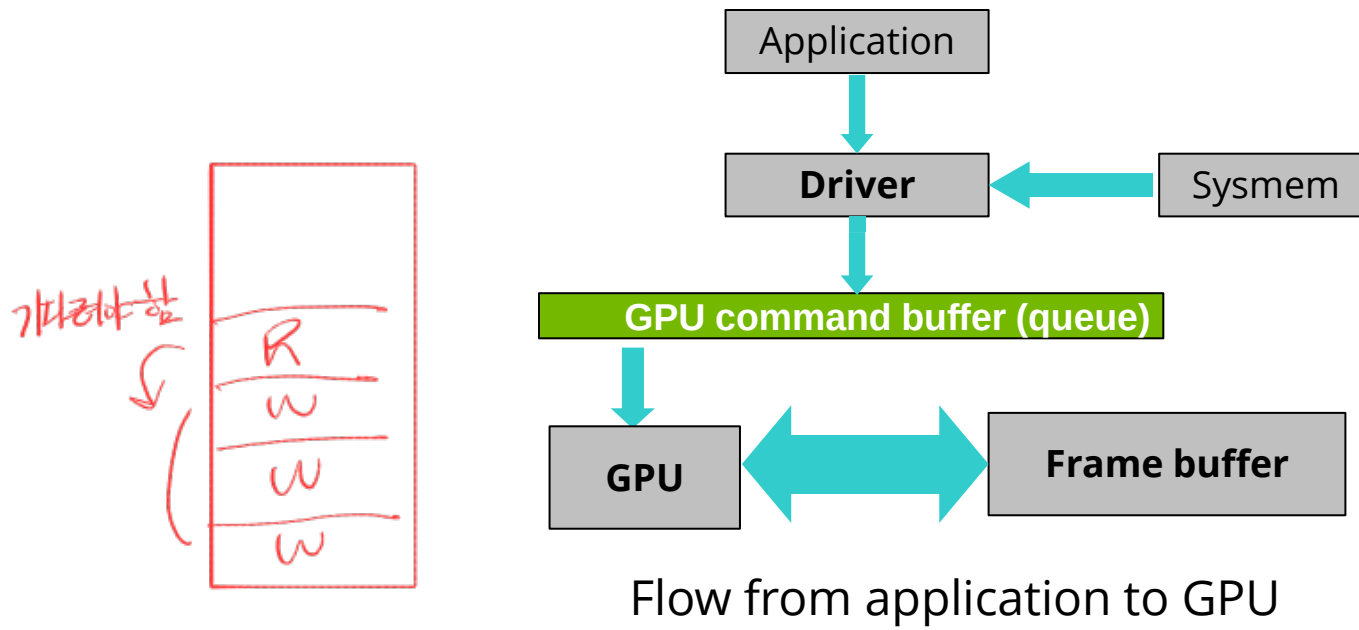
- **In practical scenarios, color buffers are solely important.**

- While color/depth buffers are still actively used, stencil buffers are not that common in modern-style GL. *잘 안 씀*
- Even reading depth buffers is rare nowadays, because we can write our own depth outputs to the color buffers (e.g., RGBZ format). *depth*

Reading from Frame Buffers

• Read operations

- While write operations are simply performed by draw calls by default, read operations can be called but take much longer than write.
- This results from asynchronous threading model of OpenGL.
- You can correctly read frame buffers after executing all the pending GPU commands in the GPU command queue.



Post-Fragment Operations and Compositing

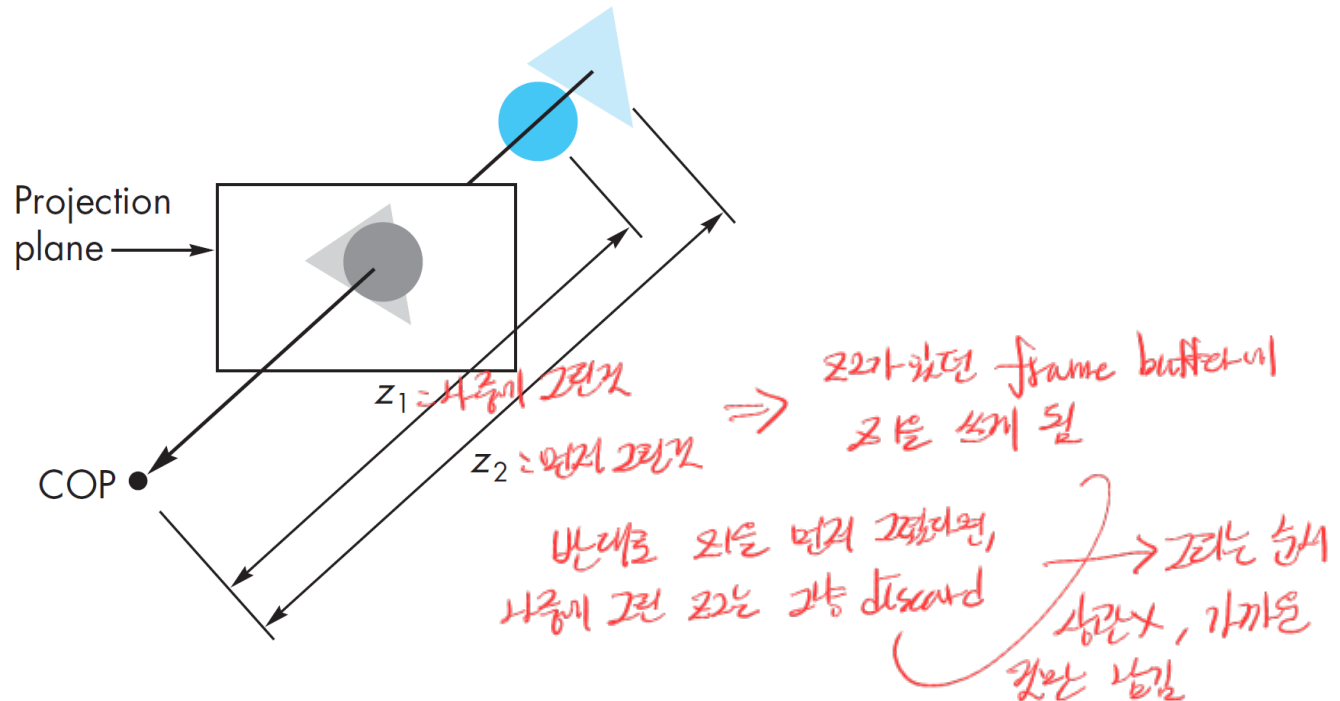
Post-Fragment Operations

- **Depth buffering**
 - maintains only nearest fragments in the buffer.
- **Linear blending**
 - alpha blending: use alpha channel
- **Logical operations: bitwise AND, OR, XOR, ...**
 - In this case, integer-format frame buffers should be used.
 - Advanced operations such as on-the-fly voxelization often use this.

Depth Buffering

- **Depth test/buffering (Z-test or Z-buffering)**

- This is the most common operation in the raster graphics.
- When enabling depth test, a fragment's depth is compared against the depth in the frame buffer. *fragment shader의 output → frame buffer에 저장*
- When the fragment depth is smaller than the existing depth, the fragment is written to the color buffer with its depth value to the depth buffer. *fragment shader의 output → frame buffer에 저장*
- Otherwise, the fragment is discarded.



Blending in Frame Buffers

- **Alpha blending**

- Linearly combines the fragment output (with its alpha value) and the colors in the frame buffer (with its alpha value).
- Source: fragment output values (S_{rgb}, S_a)
- Destination: framebuffer values (D_{rgb}, D_a)

RGB + A

이것을 control 할

$\rightarrow D_{rgb}, D_a$ 를 만든다

각각의 control X

- **Blending is specified in the host application.**

host에서 control

- Framebuffer values are not visible in fragment shaders.
- So, we tell OpenGL how to blend the fragment and the frame buffer values in the host application.
- The allowed operations are mostly linear blending, and we can specify only a limited set of the linear blending operations.

read만 가능

이렇게 blend 하는 것

제일 많이 쓰는

Opacity and Transparency

Alpha 불투명도

투명도

- **Alpha values (the last 'A' component in RGBA) now used**

- "A" (alpha) means opacity.
- Translucency = 1 - opacity (or alpha)

불투명도 투명, 반투명

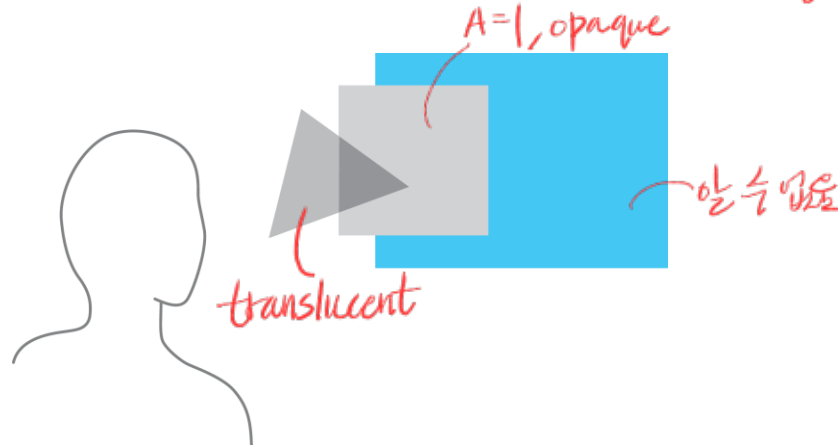
alpha가 0이면 → 투명

1이면 → 빛이 통과x, 뒤의 물체가 안보임

- **General idea on transparency**

- Opaque surfaces ($A=1$) permit no light to pass through.
- Transparent surface ($A=0$) permit all lights to pass.
- Translucent surface ($0 < A < 1$) permit pass some light.

중간이 생기게 됨



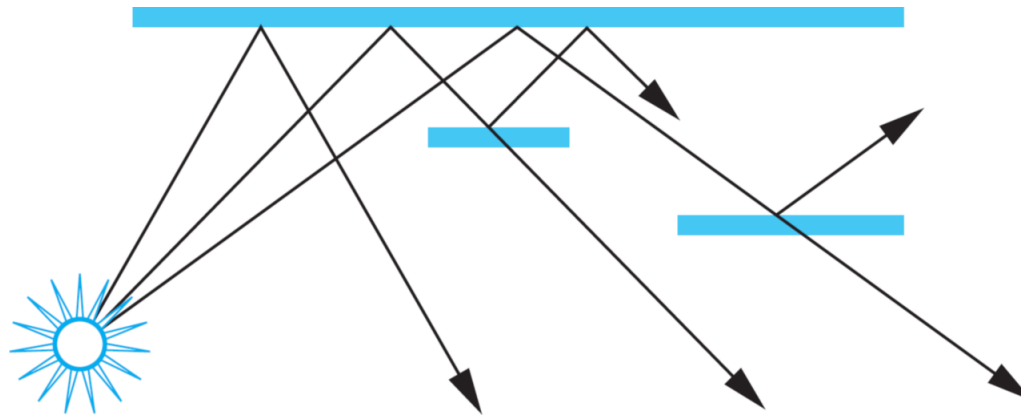
Translucent and opaque polygons

Translucency: Physical Models

depth buffering에서는 모든 object를
opaque하다고 가정해서 순서 신경X

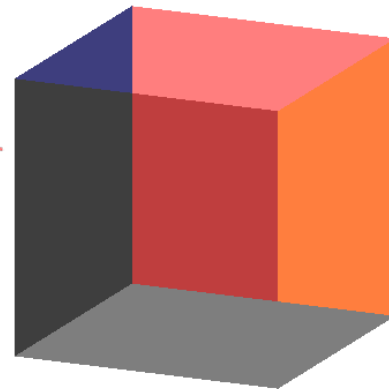
- **Translucency is a challenge in rendering.**

- Dealing with translucency in a physically correct manner is difficult due to the complexity of the internal interactions of light and materials.
- In particular, the order of rendering matters.



Scene with translucent objects

Order Dependency



그림을 렌더링할 때
순서가 중요하다

- **Is this image correct? Probably not...**

- Recall that polygons are rendered in the order the application generated.
- However, blending functions are order-dependent.
- Order-independent transparency is still an on-going research topic.

→ 이걸 안, sorting을 fragment에서 하고 453

- **Back-to-front blending**

- We need to respect the spatial orders of objects.
- Translucent objects are sorted, and then, are blended (or rendered) in a back-to-front order.

뒤에서부터 blend



Blending for Transparency

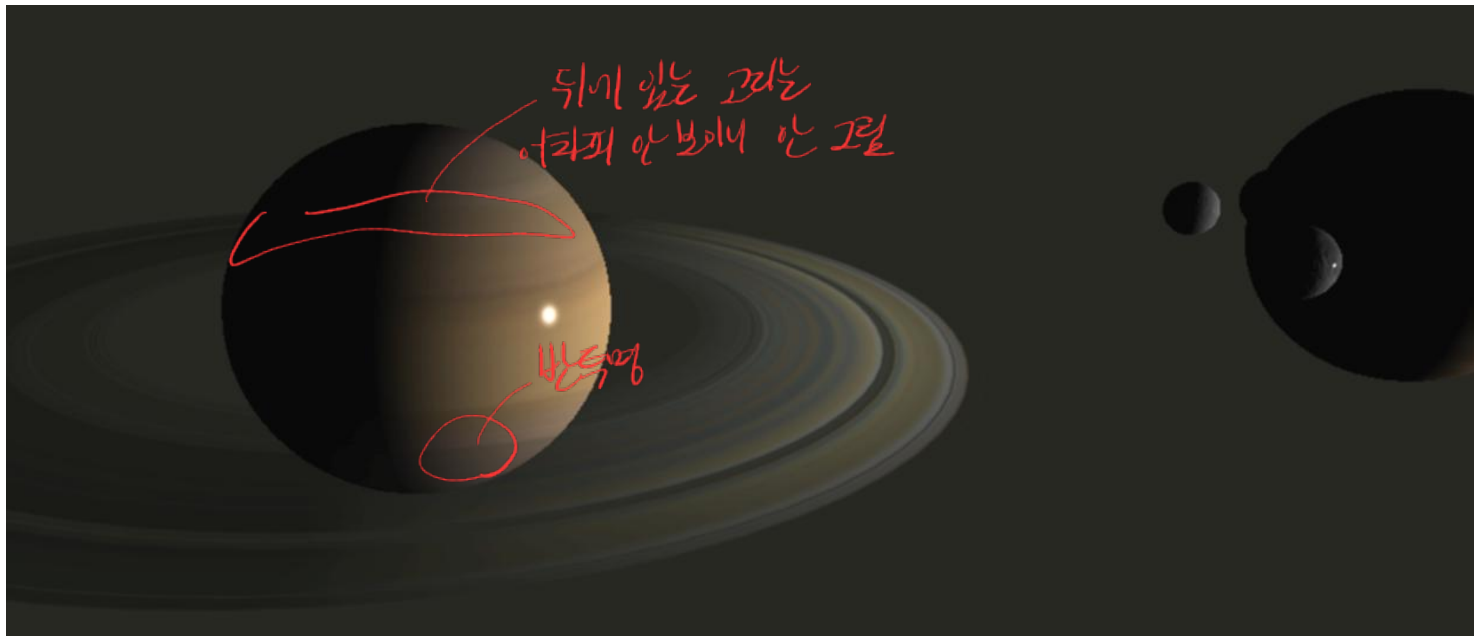
- **In practice, few objects can be approximated.**

- We first render opaque objects into the frame buffer, and then
- render translucent objects and blend them with the frame buffer.

모든 물체를 sorting X
반투명한 것만 sorting 하고
그때부터 가장 blending

- **Example: the ring system of the Saturn**

- Blend/render translucent rings in a back-to-front order.



Blending for Transparency

- Simplest alpha blending

이전 S_{rgb} 가 저장된 frame buffer를 그대로 쓸, 즉 투명

source = fragment

Destination = frame buffer

fragmented alpha가 아닌 이 되기 때문에 각 값을

$$\begin{aligned} D'_{rgb} &= S_{rgb} * S_a + D_{rgb} * (1 - S_a) \\ D'_a &= S_a * S_a + D_a * (1 - S_a) \end{aligned}$$

D_{rgb} (frame buffer에 저장)

- OpenGL example

```
glEnable( GL_BLEND ); blend를 켜
glBlendFunc( GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA );
...
```

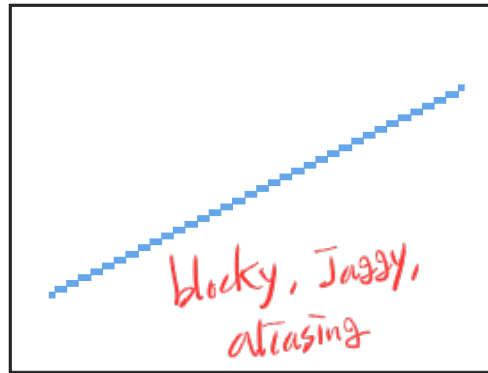
이렇게 blend하려고 전달

Blending for Transparency

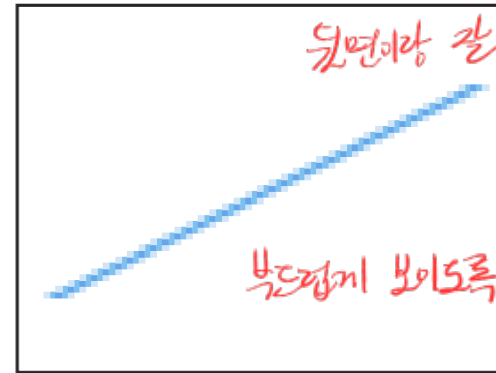
- **Another example with antialiasing**

- Point/line/polygon are smoothed for antialiasing.
- The coverage factors to neighboring pixels are computed, and alpha-blended.

line을 2픽셀씩 anti-aliasing이
필요한데 blending으로 구현됨



original jaggy line



anti-aliased line

- OpenGL example:

```
glEnable( GL_LINE_SMOOTH );  
glEnable( GL_BLEND );  
glBlendFunc( GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA );
```


Additive Blending

- **Additive/accumulative blending**

frame buffer는 2배로 두기
source에서 주어진 alpha를 weighted한
color를 더해줌

$$D'_{rgb} = S_{rgb} * S_a + D_{rgb} * \textcircled{1}$$

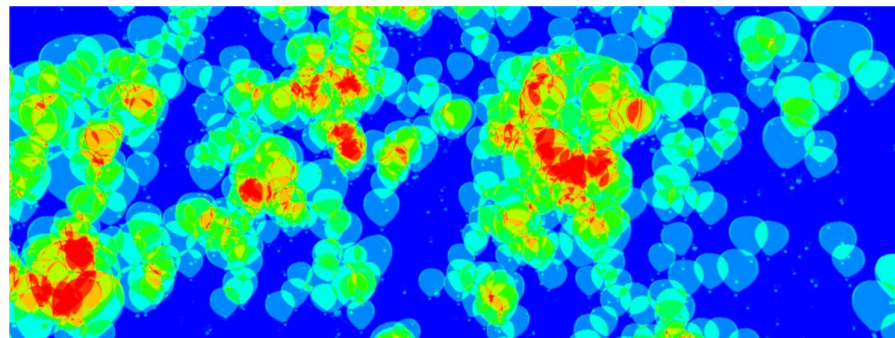
$$D'_a = S_a * S_a + D_a * \textcircled{1}$$

이렇게 계속 더해줌

- **Example**

- Counting of the number of drawn fragments

일단 depth에서 계산하여 fragment 수를 count



Color-mapped number of incoming fragments

Compositing

blending이 아님, post-fragment operational 아님
→ 2번이나 blending 작업 동작

blending은 frame buffer와
shader에서 host-application에서
control 해야 하는데, destination은
앞에 있던 shader에서 작업 blending
⇒ compositing

• Compositing

- When you are blending pre-rendered textures, you can easily implement blending in your shaders.
framebuffer는 알 수 없이 미리 렌더링한 texture 사용
- This is called the compositing rather than blending.
→ shader에서 작업 (Host에서는 렌더 작업x)
- Compositing is common in many UI/web rendering engines.
- Try to distinguish the compositing from blending.
★ 섞을 걸 미리 들고 있을 때,
Host에서 control 하려 않고 shader에서
작업 섞음 (blending과 내용은 같지만,
주변이 다름)

• Examples of the compositing

- You have background textures. (이미 그려놓은 것)
- You have foreground objects with alpha (e.g., leaves of a tree) sprite
- You can composite the background and foreground objects using alpha blending equations, while disabling blending.



Frame Buffer Objects (FBOs)

display와 연결되지 않은 frame buffer

Frame Buffer Objects (FBOs)

frame buffer → GPU

FBO → texture 사용

texture가 framebuffer처럼 사용

⇒ 둘 다 GPU에 있는 (VRAM memory 내)에
무엇이든 할 수 있는 것

- **Textures can be used as frame buffers,**
 - because they reside inside the physically same memory areas.

- **We perform the same renderings, yet with our own frame buffer (objects).**

다른 frame buffer, FBO로 렌더링

- We need to specify additional frame buffers whose render targets are attached to textures to replace the default frame buffers.
- Such frame buffers are called the frame buffer objects.

다른 target만 할 수 있는 것

Render
to texture

Render-to-Texture (RTT)

frame buffer objects texture에 그림(일들 수 있음)

- **Frame buffer objects allow us to perform “render-to-texture.”**

- The rendering results are now stored in the texture memory.
- We may read them back into main memory for further uses.
- We may use them as if a static texture in shaders.
- RTT is a primitive for many postprocessing.

texture에 rendering 결과를 그림
→ main memory로 가져와서, texture
포함 사용가능 (특수효과에 쓰임)

post-processing

- **Example: defocus blur** Camera의 blur 효과

- Pass 1: render the objects' RGB colors and depths (카메라에서 렌더링 결과) → FB → texture
- Pass 2: apply a box blur by the distance to the focal plane depth.

depth가 근원 blur ↑ ↓
(이전 렌더링 결과와 depth를 참조함)
RGBZ

Render-to-Texture: Example Applications

Deferred Rendering

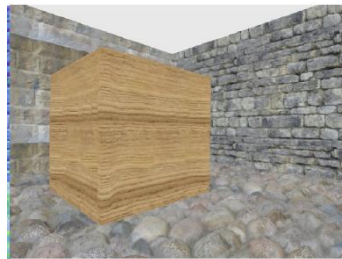
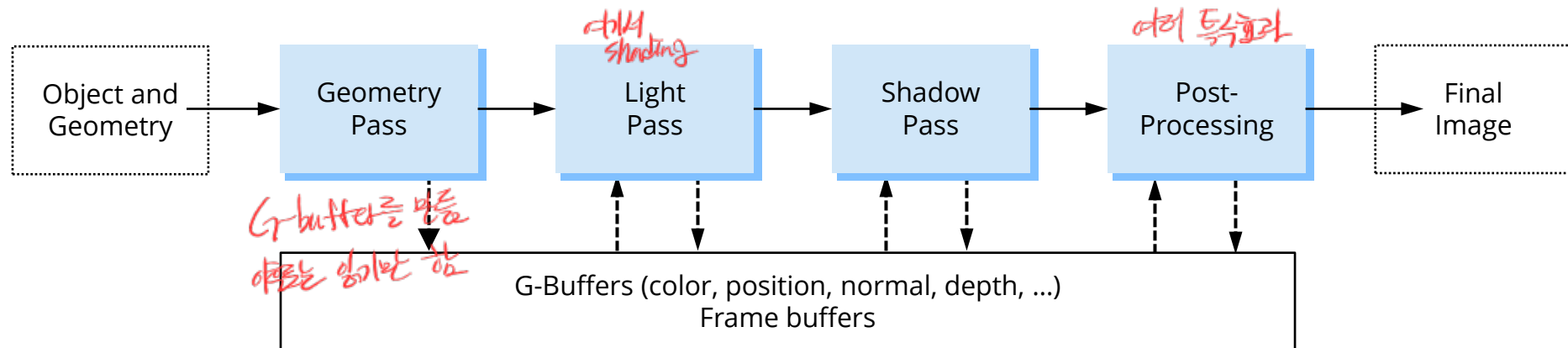
각각의 버퍼는 single pass

1. depth test를 위해 early Z
보이는 것은 shading하겠다. (shading 부하)

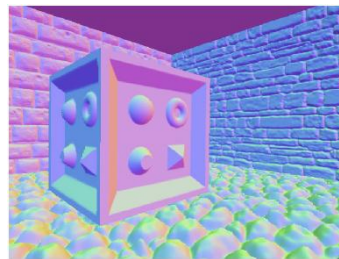
shading은 두 번 버퍼

• General idea

- Multi-pass approach yet with a single geometry pass *draw call은 한 번만 부른*
- Render attributes of the geometry and its material first on geometry pass. *color, position, normal, material...*
 - The buffer is called the **G-buffer**. *attribute를 저장 (texture와 G-buffer라고 부름)*
 - Perform shading and postprocessing later after the geometry pass *여기 두 번*



Color



Normal



Depth

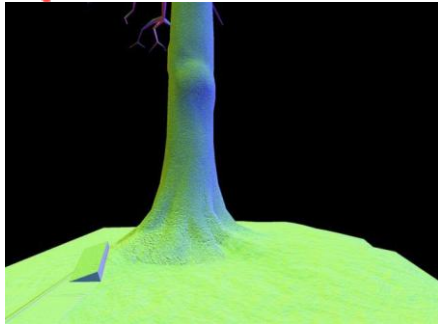
[Thaler 2012]

Examples

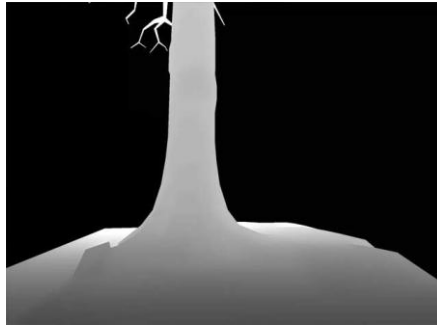
G-buffer

2차원 값이 많음 → MRT
RT는 texture array에 사용

Deferred Shading, Shawn Hargreaves, GDC 2004



normals XYZ



depth Z



specular intensity



specular power



Diffuse color (albedo) RGB



Deferred lighting result

Shadow Mapping

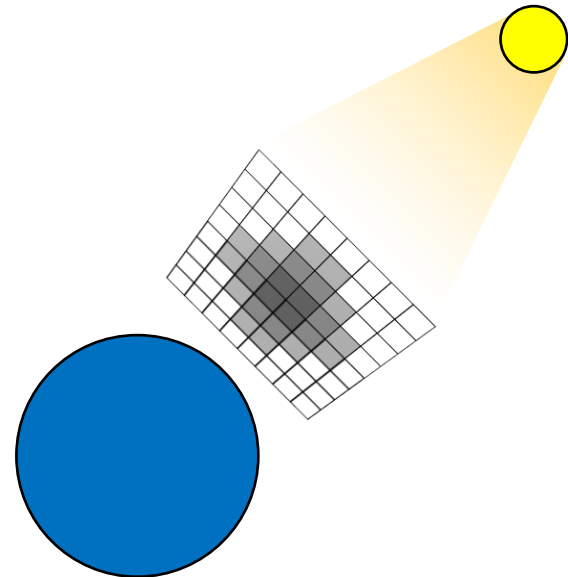
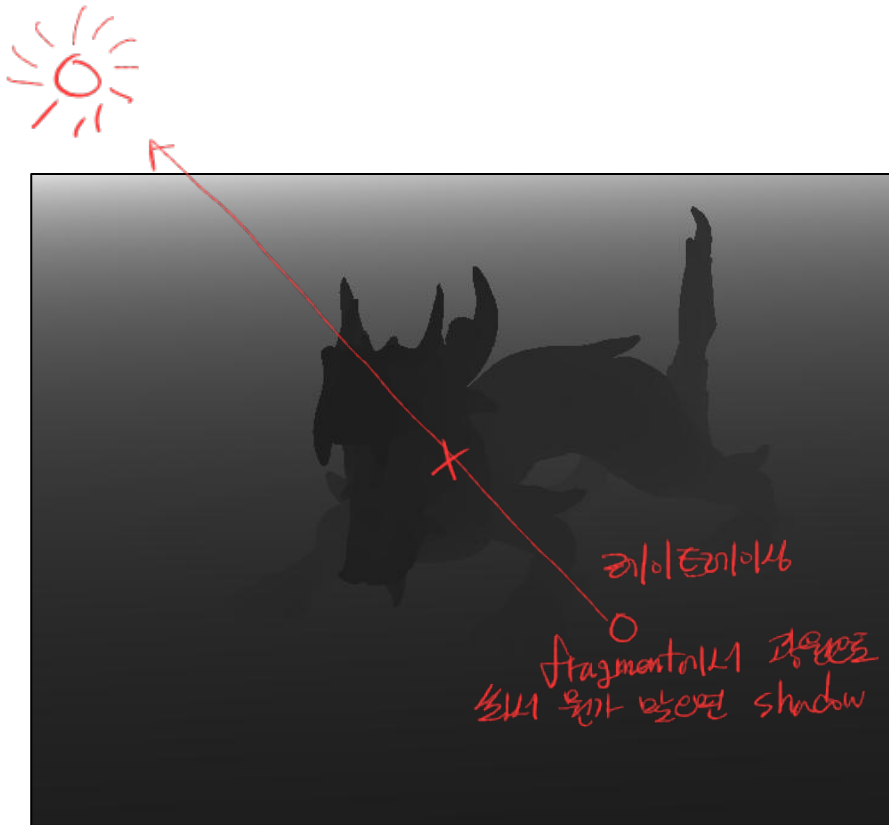
- **First pass: shadow map rendering**

- Render the depth image (shadow map) from the light viewpoint.

camera를 light로 생각하고

light에서 shadow map을 그림

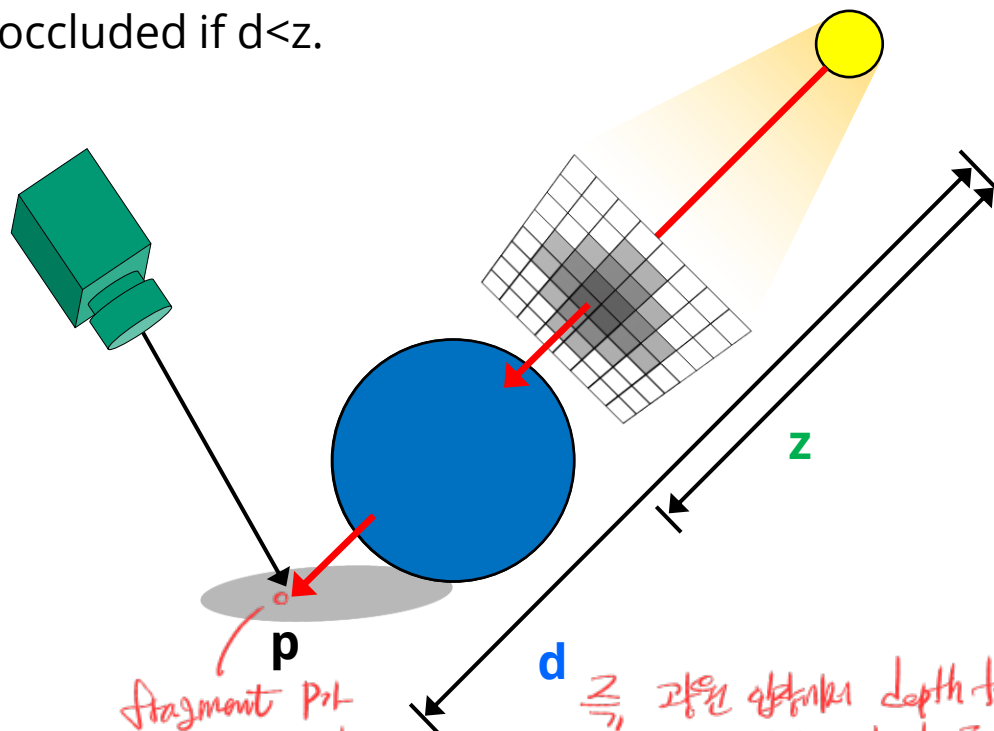
light에서 볼 depth를 저장한 texture



Shadow Mapping

- **Second pass: calculating shadow factors**

- Calculate shadow factors for each fragment that determines whether the fragment is shaded.
 - Compare depth values of the current fragment (z) and the shadow map (d).
 - Point p is lit (no shadows) if $d \geq z$.
 - Point p is occluded if $d < z$.



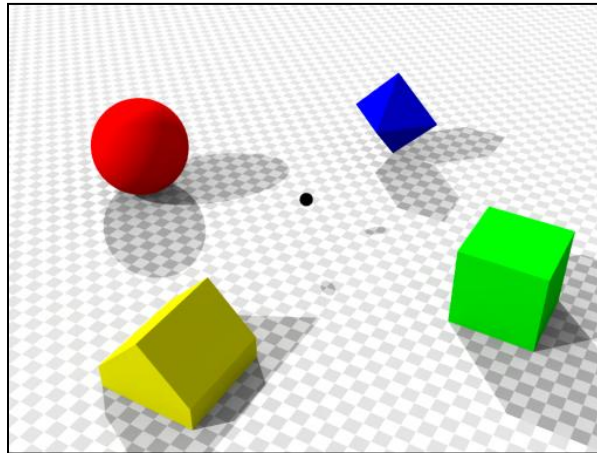
Fragment p 가
더 멀리 있으면 shadow
(shadow map보다 뒤)

즉, 광원 앞쪽의 depth test에서
discard 되는 것은 shadow를 판단

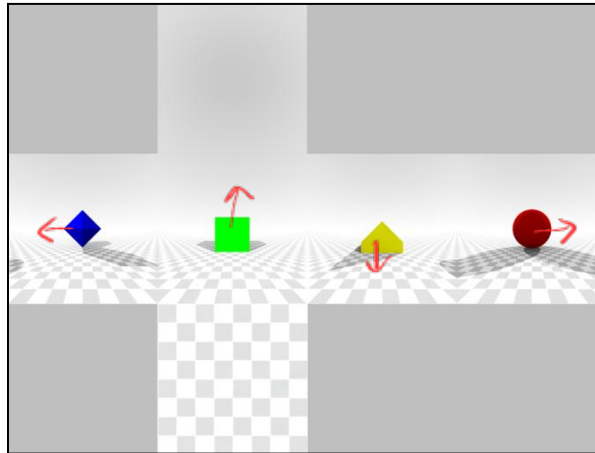
Online Cube-Map Rendering

- **Cube mapping:**

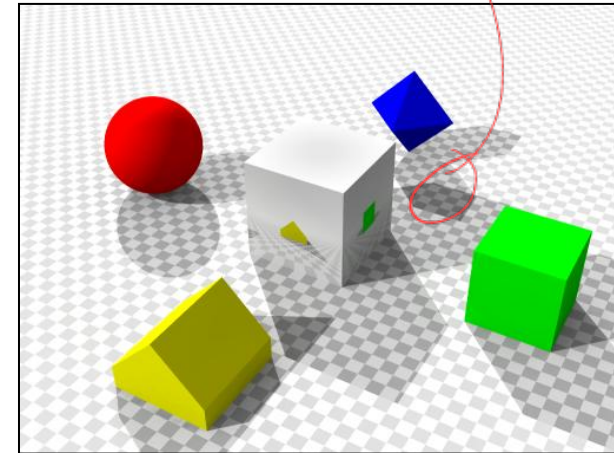
- Environment mapping via a 6-face cube map
- Could be a static texture or dynamically-generated for every frame



place a viewer in a scene



render 6-face cube map



apply texture to the surface

모든 object가 움직인다면?
매번 cubemap을 새로 그려야 될
(camera 매 프레임 texture를 새로)
FBO를 render-to-texture가 필요한 이유